claude-windows / 2ba49afa-99f4-41f2-86a2-27a289d432f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2ba49afa-99f4-41f2-86a2-27a289d432f3\subagents\workflows\wf_268ea95a-a09\agent-a78095d573984bd2a.jsonl`
- SHA-256: `9a708648e9debdabac4944c365875d7b6f0ffd7aa82b203c96472db89cfbeb3f`
- Source modified: `2026-06-21T15:33:30+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_268ea95a-a09`
- session_id: `2ba49afa-99f4-41f2-86a2-27a289d432f3`

Transcript

1. user (2026-06-21T15:29:10.462Z)

Review for ACCESSIBILITY + CORRECTNESS. A8 under C:/Users/avidu/Projects/khelsutra/web/src: background-processing reassurance + status as icon+text chips (never icon-only).
- NEW components/StatusChip.tsx: <StatusChip icon label tone={info|error}/> (icon aria-hidden, label is the text) + exported <Spinner/>.
- IngestPanel.tsx: working state -> StatusChip(info, Spinner, "Processing your video?[· phase]") + a reassurance ingest.reassure ; error state -> StatusChip(error, "?", ingest.statusFailed) above the message; removed its local Spinner (now imported).
- CompileResult.tsx: NEW building branch -> a card with StatusChip(info, Spinner, result.building) + result.buildingReassure ; done/error headers gain an inline ?/? (aria-hidden) paired with result.ready/result.failed.
- Catalogs (en/kn/hi): +ingest.reassure, +ingest.statusFailed, +result.building, +result.buildingReassure.
141 web tests green; tsc + build clean.

Read components/{StatusChip,IngestPanel,CompileResult}.tsx + the catalogs.
1. ICON+TEXT / a11y: every status must pair an icon with a real text label, and color must NOT be the only signal. Confirm the icon spans are aria-hidden and the label text carries meaning (so a screen reader announces the status). Flag any icon-only indicator. Is the aria-live region still correct for the new chip markup (ingest-status / the building card)?
2. KEY-PARITY: confirm ingest.reassure, ingest.statusFailed, result.building, result.buildingReassure exist in ALL THREE catalogs (en/kn/hi) so the hygiene key-parity test stays green. List any asymmetry.
3. NO RAW KEYS: would i18n.test.tsx's "no raw keys in DOM" assertion catch a leak? The result.* keys only render during building/done/error ? confirm they render their VALUES not the key.
4. REGRESSION: does CompileResult still return null on idle? Does the building branch correctly NOT show a dismiss button? Does the done branch still render the video+download and error branch the message? Any TS narrowing issue after splitting building out? Is the removed local Spinner fully replaced (no dangling import/use)?
Be precise (file:line). Empty findings if clean.

2. assistant (2026-06-21T15:29:14.389Z)

I'll review the A8 accessibility changes. Let me start by reading the relevant files.

3. user (2026-06-21T15:29:15.582Z)

501c014 feat(web): A5 ? friendly engine-error mapping across build + load (shared errors.*) (#62)
95195a5 feat(web): A4 ? mobile touch targets (?44px) + visible tooltip text (#61)
ccdcfcd feat(web): A1 ? plain-language copy pass on the en catalog (de-jargon) (#60)
a20002f docs: propose Multilingual Expansion ? 7 new locales (th/id/da/es/zh-Hans/ja/ko) (#58)
a0475bc docs: hygiene refresh ? sync trackers to shipped reality (#59)
---BRANCH---
feat/a8-status-chips

4. assistant (2026-06-21T15:29:18.840Z)

I'm on the right branch. Let me read all the files in question.

5. user (2026-06-21T15:29:19.760Z)

```
1       // A status indicator that is ALWAYS icon + text ? never icon-only (HCI4D guidance for
non-tech-savvy
2       // users, FRIENDLY_AND_MULTILINGUAL §4: pair every icon with a word, don't rely on color
alone). Used
3       // for the ingest "working / couldn't process" states and the build "stitching / ready /
failed" states.
4       import type { ReactNode } from "react";
5
6       type Tone = "info" | "error";
7
8       const TONE: Record<Tone, string> = {
9         info: "bg-ink/5 text-ink/70",
10        error: "bg-red-100 text-red-800",
11      };
12
13      interface Props {
14        /** A glyph or the <Spinner/> ? decorative; the label carries the meaning for screen
readers. */
15        icon: ReactNode;
16        label: string;
17        tone?: Tone;
18      }
19
20      export function StatusChip({ icon, label, tone = "info" }: Props) {
21        return (
22          <span className={`inline-flex items-center gap-1.5 rounded-full px-3 py-1 text-sm
font-medium ${TONE[tone]}`}>
23            <span aria-hidden="true" className="inline-flex">
24              {icon}
25            </span>
26            <span>{label}</span>
27          </span>
28        );
29      }
30
31      /** Indeterminate spinner ? the engine reports coarse phase WORDS, not a percentage, so
progress is a
32       *  spinner + the phase label rather than a misleading progress bar. */
33      export function Spinner() {
34        return (
35          <span
36            className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-
ink/20 border-t-ink/70"
37            aria-hidden="true"
38          />
39        );
40      }
41
```

6. user (2026-06-21T15:29:20.317Z)

```
1       // The front-half ingest screen (appliance S1/S2, tracker B-P2). Point the tool at a
video ? a
2       // `gs://` bucket URI or a path on the engine host ? and it submits to POST
/api/process, shows
3       // live progress while polling GET /api/jobs/{id}, and hands the resulting video_id to
the existing
4       // rally-selection back-half (App lifts ?video=<id>). Honest by construction: the engine
reports
```

```
5       // coarse phase WORDS (not a %), and a success-shaped failure (done but no rallies) is
shown, never
6       // silently treated as a load.
7       import { useState } from "react";
8       import type { FormEvent } from "react";
9       import { useTranslation } from "react-i18next";
10      import { useIngestJob } from "../hooks/useIngestJob";
11      import { errorText } from "../lib/errorText";
12      import { StatusChip, Spinner } from "./StatusChip";
13
14      interface Props {
15        /** Called with the engine's video_id once a submitted job finishes successfully. */
16        onLoaded: (videoId: string) => void;
17      }
18
19      export function IngestPanel({ onLoaded }: Props) {
20        const { t } = useTranslation();
21        const [uri, setUri] = useState("");
22        const { phase, submit, cancel, reset, busy } = useIngestJob(onLoaded);
23
24        const onSubmit = (e: FormEvent) => {
25          e.preventDefault();
26          submit(uri);
27        };
28
29        // Shared mapping with the build flow (lib/errorText): `engine` failures carry the
engine's own
30        // message (a 400 scheme/path detail, a worker error); every other code maps to a
friendly
31        // localized line ? the network case especially, since this is the first screen a
fresh user hits
32        // and "is the engine running?" must read plainly.
33        const errorMsg = phase.status === "error" ? errorText(t, phase.error) : null;
34
35        return (
36          <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
37            <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
38            <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
39
40            <form onSubmit={onSubmit} className="mt-3 flex flex-wrap items-center gap-2">
41              <input
42                type="text"
43                value={uri}
44                onChange={(e) => setUri(e.target.value)}
45                placeholder={t("ingest.uriPlaceholder")}
46                aria-label={t("ingest.uriAriaLabel")}
47                disabled={busy}
48                className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15 bg-
white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
49              />
50              <button
51                type="submit"
52                disabled={busy}
53                className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
54              >
55                {phase.status === "submitting" ? t("ingest.submitting") : t("ingest.submit")}
56              </button>
57              {busy && (
58                <button
59                  type="button"
60                  onClick={cancel}
61                  className="min-h-11 rounded-xl border border-black/15 px-4 py-2 text-sm
hover:border-green"
```

```
62                >
63                  {t("ingest.cancel")}
64                </button>
65              )}
66            </form>
67
68            <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
69
70            {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
71                (A8). Indeterminate: the engine reports phase words ("hashing", "segmenting"),
not a %. */}
72            <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
73              {busy && (
74                <div>
75                  <StatusChip
76                    tone="info"
77                    icon={<Spinner />}
78                    label={
79                      t("ingest.working") +
80                      (phase.status === "polling" && phase.progress
81                        ? ` · ${t("ingest.progress", { progress: phase.progress })}`
82                        : "")
83                    }
84                  />
85                  {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
86                  <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
87                </div>
88              )}
89              {phase.status === "error" && (
90                <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
91                  <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
92                  <p className="mt-2">{errorMsg}</p>
93                  <button
94                    type="button"
95                    onClick={reset}
96                    className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
97                  >
98                    {t("ingest.retry")}
99                  </button>
100               </div>
101             )}
102           </div>
103         </section>
104       );
105     }
106
```

7. user (2026-06-21T15:29:20.574Z)

```
1       // The compiled-reel result (PER_RALLY_SELECTION.md P5): once Build ? POST /api/compile
succeeds,
2       // preview the reel inline (<video> over GET /api/highlights/...) and offer a download.
The engine
3       // returns a server-local filepath, NOT a URL (main.py:394), so the SPA builds the URL
itself. On a
4       // failure (e.g. the min_shot_count 400) we show the engine's reason ? loud, never
swallowed.
5
6       import { useTranslation } from "react-i18next";
7       import { StatusChip, Spinner } from "./StatusChip";
8
```

```
      export type BuildState =
        | { status: "idle" }
        | { status: "building" }
        | { status: "done"; url: string; filename: string }
        | { status: "error"; error: string };

      interface Props {
        build: BuildState;
        onDismiss: () => void;
      }

      export function CompileResult({ build, onDismiss }: Props) {
        const { t } = useTranslation();
        if (build.status === "idle") return null;

        // Build state (A8): a visible "stitching?" card with an icon+text chip + reassurance,
so the user
        // isn't staring at a frozen footer wondering whether anything is happening.
        if (build.status === "building") {
          return (
            <section data-testid="compile-result" className="mt-6 rounded-xl border border-
black/10 bg-paper p-4" aria-live="polite">
              <StatusChip tone="info" icon={<Spinner />} label={t("result.building")} />
              <p className="mt-2 text-sm text-ink/60">{t("result.buildingReassure")}</p>
            </section>
          );
        }

        const isError = build.status === "error";
        return (
          <section
            data-testid="compile-result"
            className={"mt-6 rounded-xl border p-4 " + (isError ? "border-red-200 bg-red-50" :
"border-green/30 bg-green/5")}
          >
            <div className="flex items-start justify-between gap-3">
              {/* Status as icon+text (never icon-only): ? ready / ? failed (A8). */}
              <h2 className="flex items-center gap-2 font-semibold text-ink">
                <span aria-hidden="true" className={isError ? "text-red-700" : "text-
green"}>{isError ? "?" : "?"}</span>
                {build.status === "done" ? t("result.ready") : t("result.failed")}
              </h2>
              <button
                type="button"
                onClick={onDismiss}
                aria-label={t("result.dismiss")}
                className="flex h-11 w-11 shrink-0 items-center justify-center rounded-full
text-ink/50 hover:text-ink"
              >
                ?
              </button>
            </div>

            {build.status === "done" ? (
              <div className="mt-3">
                {/* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */}
                <video
                  data-testid="reel-video"
                  src={build.url}
                  controls
                  playsInline
                  className="w-full max-w-xl rounded-lg bg-black"
                />
                <a
```

```
68                  href={build.url}
69                  download={build.filename}
70                  className="mt-3 inline-flex min-h-11 items-center rounded-full bg-green px-4
py-2 text-sm font-semibold text-[#04140D]"
71                >
72                  {t("result.download", { filename: build.filename })}
73                </a>
74              </div>
75            ) : (
76              <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
77                {build.error}
78              </p>
79            )}
80          </section>
81        );
82      }
83
```

8. assistant (2026-06-21T15:29:23.472Z)

Now let me read the three catalogs to verify key parity.

9. user (2026-06-21T15:29:25.678Z)

```
hygiene.test.ts
i18n.test.tsx
index.ts
locales
---
---globbing---
```

10. user (2026-06-21T15:29:25.925Z)

```
No files found
```

11. user (2026-06-21T15:29:29.200Z)

```
total 4
drwxr-xr-x 1 avidu 197609 0 Jun 21 12:43 .
drwxr-xr-x 1 avidu 197609 0 Jun 21 15:45 ..
drwxr-xr-x 1 avidu 197609 0 Jun 21 20:55 en
drwxr-xr-x 1 avidu 197609 0 Jun 21 20:56 hi
drwxr-xr-x 1 avidu 197609 0 Jun 21 20:55 kn
```

12. user (2026-06-21T15:29:33.033Z)

```
C:/Users/avidu/Projects/khelsutra/web/src/i18n/locales/en/:
total 8
drwxr-xr-x 1 avidu 197609    0 Jun 21 20:55 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:43 ..
-rw-r--r-- 1 avidu 197609 4504 Jun 21 20:55 common.json

C:/Users/avidu/Projects/khelsutra/web/src/i18n/locales/hi/:
total 12
drwxr-xr-x 1 avidu 197609    0 Jun 21 20:56 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:43 ..
-rw-r--r-- 1 avidu 197609 8632 Jun 21 20:56 common.json

C:/Users/avidu/Projects/khelsutra/web/src/i18n/locales/kn/:
total 12
drwxr-xr-x 1 avidu 197609     0 Jun 21 20:55 .
drwxr-xr-x 1 avidu 197609     0 Jun 21 12:43 ..
-rw-r--r-- 1 avidu 197609 10058 Jun 21 20:55 common.json
```

13. user (2026-06-21T15:29:36.364Z)


```
1       {
2         "app": {
3           "badge": "pick your rallies · alpha",
4           "title": "Pick the rallies for your reel",
5           "intro": "Filter by <b>rally length</b> ? e.g. <i>?top 15 longest?</i> ? then fine-
tune by tapping cards. Everything starts <b>selected</b>; just drop the ones you don't want."
6         },
7         "lang": {
8           "label": "Language",
9           "en": "English",
10          "kn": "?????",
11          "hi": "??????"
12        },
13        "bulk": {
14          "selectAll": "Select all",
15          "clear": "Clear",
16          "invert": "Invert"
17        },
18        "mode": {
19          "demo": "These are sample rallies ? process a video above to work on a real match.",
20          "loading": "Finding the rallies in your video?",
21          "error": "Couldn't load this video's rallies: {error}",
22          "loaded": "Found <b>{count}</b> rallies in your video."
23        },
24        "query": {
25          "placeholder": "e.g. over 10s · top 15 longest · shortest first",
26          "ariaLabel": "Filter rallies by length, count or order",
27          "apply": "Apply",
28          "notAvailable": "not available yet",
29          "notAvailableTitle": "?{token}? needs {kind} detection ? not available yet",
30          "notAvailableHelp": "Shot type, players and score aren't available yet ? filter by
rally length instead.",
31          "unreadable": "Couldn't read that ? try ?over 10s? or ?top 15 longest?.",
32          "hint": "Type over 10s for rallies longer than 10 seconds ? or tap a sample below.",
33          "try": "Sample queries:",
34          "support": "Filtering by <b>rally length</b> is fully supported. Shot-count is
<x>experimental</x> (under development)."
35        },
36        "grid": {
37          "empty": "No rallies to show yet.",
38          "rally": "Rally {n}",
39          "inReel": "Rally {n}, in reel",
40          "notInReel": "Rally {n}, not in reel",
41          "start": "start",
42          "long": "long",
43          "shots": "~{count} shots · experimental",
44          "shotsTitle": "Shot count is experimental (under development) and may be
inaccurate{conf}. Filter by rally length instead.",
45          "shotsConfidence": " ? confidence: {conf}"
46        },
47        "footer": {
48          "count": "{count, plural, one {# rally} other {# rallies}}",
49          "summaryRest": "selected · {total} total",
50          "reelName": "Reel name",
51          "reelNamePlaceholder": "my-highlights",
52          "build": "Build reel ({count})",
53          "building": "Building?",
54          "floorOne": "<b>Your only selected rally</b> has fewer than {min} shots ? it would
be skipped, so the build would fail. Pick a longer rally.",
55          "floorAll": "<b>All {count} selected rallies</b> have fewer than {min} shots ?
they'd all be skipped, so the build would fail. Pick longer rallies.",
56          "floorSomeLead": "{count, plural, one {# rally} other {# rallies}}",
57          "floorSomeRest": "with fewer than {min} shots will be skipped when building (they're
```

```
too short)."
58        },
59        "result": {
60          "ready": "Your reel is ready",
61          "failed": "Build failed",
62          "building": "Stitching your reel?",
63          "buildingReassure": "Joining your selected rallies into one video ? this usually
takes under a minute.",
64          "dismiss": "Dismiss",
65          "download": "Download {filename}"
66        },
67        "build": {
68          "noVideo": "These are sample rallies ? process a real video first to build a reel."
69        },
70        "ingest": {
71          "title": "Process a video",
72          "intro": "Point KhelSutra at a video and it finds the rallies for you ? this can
take a few minutes.",
73          "uriPlaceholder": "gs://your-bucket/match.mp4",
74          "uriAriaLabel": "Video location ? a gs:// bucket URL or a path on the engine host",
75          "hint": "Paste a cloud-storage link (gs://?), or a path to a video on the machine
running KhelSutra.",
76          "submit": "Process video",
77          "submitting": "Submitting?",
78          "cancel": "Cancel",
79          "working": "Processing your video?",
80          "progress": "{progress}",
81          "retry": "Try again",
82          "reassure": "Safe to wait ? KhelSutra keeps working even if your connection is slow;
a long video can take several minutes.",
83          "statusFailed": "Couldn't process"
84        },
85        "errors": {
86          "networkError": "Can't reach KhelSutra. Is it running?",
87          "invalidUri": "Enter a video location first ? a gs:// link or a path on the machine
running KhelSutra.",
88          "emptyResult": "Processing finished but found no rallies in this video. Try a
clearer match recording.",
89          "timeout": "This is taking longer than expected. The job may still be running ?
reload with the video link, or try again.",
90          "engineGeneric": "Something went wrong. Please try again."
91        },
92        "selection": {
93          "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
94        }
95      }
96
```

14. user (2026-06-21T15:29:36.768Z)

```
1       {
2         "app": {
3           "badge": "?????-?????? ????? · ?????",
4           "title": "????? ????????? ???????????? ?????",
5           "intro": "<b>?????? ????</b>? ????????? ??????? ???? ? ???. <i>???? ????? ????
15?</i> ? ???? ????????????? ?????? ???? ?????????? ???????. ?????? <b>?????????</b>
???????????????; ?????????????? ????? ??????????."
6         },
7         "lang": {
8           "label": "????",
9           "en": "English",
10          "kn": "?????",
11          "hi": "??????"
12        },
13        "bulk": {
```

```
    "selectAll": "????????? ?????????",
    "clear": "???????????",
    "invert": "????????"
  },
  "mode": {
    "demo": "???? ???? ? ?????? ????? ???? ????? ???????????? ?????????????? <code>?video=?video_id?</code> ???? ????????.",
    "loading": "<code>{videoId}</code> ???? ???????????? ???? ??????????????",
    "error": "<code>{videoId}</code> ???? ???????????? ???? ??????????: {error}",
    "loaded": "? ?????????? <b>{count}</b> ???????????? ???? ?????????."
  },
  "query": {
    "placeholder": "???. over 10s · top 15 longest · shortest first",
    "ariaLabel": "????, ????? ???? ????? ????????? ???????????? ??????? ????",
    "apply": "????????",
    "notAvailable": "????? ?????????",
    "notAvailableTitle": "?{token}? ?? {kind} ????? ???? ? ????? ?????????",
    "notAvailableHelp": "???? ??????, ??????? ????? ?????? ????? ????????? ? ?????? ?????? ????? ????????? ??????? ????.",
    "unreadable": "?????? ?????????? ? ?over 10s? ???? ?top 15 longest? ??????????.",
    "hint": "10 ???????????? ????? ????????????? over 10s ???? ???? ???? ? ???? ?????? ?????????? ?????? ????.",
    "try": "????? ??????????:",
    "support": "<b>?????? ????</b>? ????????? ??????? ???????? ??????????? ?????????????. ????-????? <x>?????????</x> (???????????????)."
  },
  "grid": {
    "empty": "??????? ????? ?????? ???????????.",
    "rally": "?????? {n}",
    "inReel": "?????? {n}, ??????????",
    "notInReel": "?????? {n}, ?????????????",
    "start": "???????",
    "long": "????",
    "shots": "~{count} ???????? · ????????",
    "shotsTitle": "???? ????? ?????????????? (???????????????) ????? ?????????????{conf}. ?????? ?????? ????? ????????? ??????? ????.",
    "shotsConfidence": " ? ???????: {conf}"
  },
  "footer": {
    "count": "{count, plural, one {# ??????} other {# ????????}}",
    "summaryRest": "??????????? · ????? {total}",
    "reelName": "???? ?????",
    "reelNamePlaceholder": "my-highlights",
    "build": "???? ????? ({count})",
    "building": "???????????????",
    "floorOne": "<b>????? ???? ?????????? ??????</b>????? {min} ?????? ????? ?????????? ? ?????? ?????? ???????????????, ???????? ???? ????????????. ????? ?????????? ?????.",
    "floorAll": "<b>?????????? ????? {count} ????????????</b> {min} ?????? ????? ?????????? ? ?????? ????????????? ??????????????, ???????? ???? ????????????. ????? ???????????? ?????.",
    "floorSomeLead": "{count, plural, one {# ??????} other {# ????????}}",
    "floorSomeRest": "{min} ?????? ????? ????????????? ???????? ???????????????? (???????? <code>min_shot_count</code> ????)."
  },
  "result": {
    "ready": "????? ???? ???????????",
    "failed": "???? ??????????",
    "building": "????? ???? ???????????????",
    "buildingReassure": "???? ?????????? ???????????? ???? ???????? ????????????????? ? ??????????? ???? ?????????? ????????.",
    "dismiss": "?????????",
    "download": "{filename} ????????? ????"
  },
  "build": {
    "noVideo": "???? ?????? ????????????? ????????????????? ?video=<video_id> ???? ????????
```

```
? ???? ????????? ?????? ????????."
69        },
70        "ingest": {
71          "title": "?????? ???????????????",
72          "intro": "??????????????? ???? ???????? ?????? ? ????? ????? ???? ?????????????? ???? ? ??? ??????????? ???????????. ?????? ????? ???????? ??????????.",
73          "uriPlaceholder": "gs://your-bucket/match.mp4",
74          "uriAriaLabel": "?????? ???? ? gs:// ????? URL ???? ?????? ?????????????? ??",
75          "hint": "gs:// ????? ?????, ???? ?????? ???????????? ?????? ?????? ??????? ??????.",
76          "submit": "?????? ???????????????",
77          "submitting": "?????????????????",
78          "cancel": "???????????",
79          "working": "????? ?????? ??????????????????????",
80          "progress": "{progress}",
81          "retry": "????? ??????????",
82          "reassure": "???????? ???????? ? ????? ?????? ????????????? ?????????? ???????????? ???? ???????????????; ????? ???????? ????? ???????? ??????????.",
83          "statusFailed": "?????????????????????"
84        },
85        "errors": {
86          "networkError": "?????? ???????????????. ??? ???????????????",
87          "invalidUri": "????? ?????? ????????? ??????? ? gs:// ????? ???? ??????????????? ??.",
88          "emptyResult": "????????? ???????? ???? ? ??????????? ?????? ????????? ????????. ????????? ????? ??????????? ??????????.",
89          "timeout": "???????????? ?????? ??? ?????????????????. ???? ????? ???????????????? ? ?????? ????????????? ??????? ????, ???? ????? ??????????.",
90          "engineGeneric": "??? ?????????. ???????? ????? ??????????."
91        },
92        "selection": {
93          "count": "{count, plural, one {# ?????? ???????????} other {# ????????? ???????????}}"
94        }
95      }
96
```

15. user (2026-06-21T15:29:37.172Z)

```
1       {
2         "app": {
3           "badge": "?????-???? ??? · ??????",
4           "title": "???? ??? ?? ??? ??????? ?????",
5           "intro": "<b>???? ?? ?????</b> ?? ???? ?? ??????? ???? ? ???? <i>????? ???? ??? 15?</i> ? ??? ????? ??? ???? ?????? ?? ???????? ????? ?? ??? <b>?????</b> ???? ???? ??; ?? ?? ???? ????? ?????? ??????"
6         },
7         "lang": {
8           "label": "????",
9           "en": "English",
10          "kn": "?????",
11          "hi": "??????"
12        },
13        "bulk": {
14          "selectAll": "??? ?????",
15          "clear": "???? ????",
16          "invert": "?????"
17        },
18        "mode": {
19          "demo": "???? ???? ? ???? ???? ??? ???? ?? ??? ???? ???? ?? ??? <code>?video=?video_id?</code> ?? ??????",
20          "loading": "<code>{videoId}</code> ?? ??? ??????? ??? ?? ??? ????",
21          "error": "<code>{videoId}</code> ?? ??? ??????? ??? ???? ?? ????: {error}",
22          "loaded": "?? ???? ?? <b>{count}</b> ??????? ??? ?????"
23        },
```

```
      "query": {
        "placeholder": "???. over 10s · top 15 longest · shortest first",
        "ariaLabel": "?????, ?????? ?? ???? ?? ???? ?? ??????? ??????? ????",
        "apply": "???? ????",
        "notAvailable": "??? ?????? ????",
        "notAvailableTitle": "?{token}? ?? ??? {kind} ????? ????? ? ??? ?????? ????",
        "notAvailableHelp": "??? ??????, ??????? ?? ????? ??? ?????? ???? ??? ? ???? ????
???? ?? ????? ?? ??????? ?????",
        "unreadable": "?? ??? ???? ??? ? ?over 10s? ?? ?top 15 longest? ????????",
        "hint": "10 ????? ?? ???? ??????? ?? ??? over 10s ???? ???? ? ?? ???? ?? ??? ?????
??? ?????",
        "try": "????? ??????:",
        "support": "<b>???? ?? ?????</b> ?? ???? ?? ??????? ???? ???? ??? ??????? ???
???-????? <x>???????????</x> ?? (?????????)?"
      },
      "grid": {
        "empty": "?????? ?? ??? ??? ??? ???? ???? ???",
        "rally": "???? {n}",
        "inReel": "???? {n}, ??? ???",
        "notInReel": "???? {n}, ??? ??? ????",
        "start": "????",
        "long": "????",
        "shots": "~{count} ????? · ???????????",
        "shotsTitle": "??? ????? ??????????? ?? (?????????) ?? ??? ?? ???? ??{conf}? ????
???? ???? ?? ????? ?? ??????? ?????",
        "shotsConfidence": " ? ???????: {conf}"
      },
      "footer": {
        "count": "{count, plural, one {# ????} other {# ???????}}",
        "summaryRest": "????? · ??? {total}",
        "reelName": "??? ?? ???",
        "reelNamePlaceholder": "my-highlights",
        "build": "??? ????? ({count})",
        "building": "?? ??? ???",
        "floorOne": "<b>???? ??????? ????? ????</b> ??? {min} ?? ?? ????? ??? ? ???? ???
???? ????, ????? ????? ???? ????? ?? ???? ???? ??????",
        "floorAll": "<b>??? {count} ????? ???????</b> ??? {min} ?? ?? ????? ??? ? ???? ?? ??
?? ???? ???? ??, ????? ????? ???? ????? ???? ??????? ??????",
        "floorSomeLead": "{count, plural, one {# ????} other {# ???????}}",
        "floorSomeRest": "?????? {min} ?? ?? ????? ???, ????? ???? ??? ???? ?? ?????? (????
?? <code>min_shot_count</code> ????)?"
      },
      "result": {
        "ready": "???? ??? ????? ??",
        "failed": "????? ????",
        "building": "???? ??? ????? ?? ??? ???",
        "buildingReassure": "???? ???? ??? ??????? ?? ?? ?????? ??? ????? ?? ??? ?? ? ?????
?? ?? ???? ?? ?? ???? ???",
        "dismiss": "????? ????",
        "download": "{filename} ??????? ????"
      },
      "build": {
        "noVideo": "??? ????? ?? ??? ???? ???? ?? ??? ?video=<video_id> ?? ????? ? ???? ????
?????? ???? ?? ?????"
      },
      "ingest": {
        "title": "?????? ??????? ????",
        "intro": "???????? ?? ?? ?????? ?????? ? ?? ???? ???? ?? ???? ?? ????? ????? ? ?? ??
??????? ????? ????? ????? ??? ???? ?? ???? ????",
        "uriPlaceholder": "gs://your-bucket/match.mp4",
        "uriAriaLabel": "?????? ?? ????? ? ?? gs:// ???? URL ?? ???? ????? ?? ?? ???",
        "hint": "?? gs:// ???? ????, ?? ???? ??? ??? ???? ?? ?????? ?? ??? ????? ?????",
        "submit": "?????? ??????? ????",
        "submitting": "???? ?? ??? ???",
        "cancel": "???? ????",
```

```
79          "working": "???? ?????? ??????? ?? ??? ???",
80          "progress": "{progress}",
81          "retry": "??? ?? ????? ????",
82          "reassure": "??????? ???? ???????? ?? ? ???? ??????? ???? ???? ?? ?? ???????? ?????
?? ??? ???? ???? ??; ???? ?????? ??? ??? ???? ?? ???? ????",
83          "statusFailed": "??????? ???? ?? ???"
84        },
85        "errors": {
86          "networkError": "???? ?? ???? ????? ?? ???? ???? ?? ?? ??? ???",
87          "invalidUri": "???? ?????? ?? ????? ????? ? ?? gs:// ???? ?? ???? ?? ?? ????",
88          "emptyResult": "??????????? ???? ??? ?? ?? ?????? ??? ??? ???? ???? ????? ?? ???? ???
?????????? ????????",
89          "timeout": "?? ??????? ?? ???? ??? ?? ??? ??? ??? ??? ?? ?? ??? ?? ???? ?? ? ??????
???? ?? ??? ????? ????, ?? ??? ?? ????? ?????",
90          "engineGeneric": "??? ?????? ?? ??? ????? ??? ?? ????? ?????"
91        },
92        "selection": {
93          "count": "{count, plural, one {# ???? ?????} other {# ??????? ?????}}"
94        }
95      }
96
```

16. assistant (2026-06-21T15:29:42.042Z)

All three catalogs have the new keys. Now let me look at the test files to verify the hygiene key-parity test and the i18n "no raw keys" assertion, plus check how CompileResult is used by its parent (to confirm idle/regression behavior).

17. user (2026-06-21T15:29:42.992Z)

```
1       // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
2       //   1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3       //      drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4       //      starts enforcing the moment a kn/hi catalog lands (B5/B6).
5       //   2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6       //      aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7       //      the doc; an AST-level sweep of JSX text can extend it later.
8       import { describe, it, expect } from "vitest";
9       import { readdirSync, readFileSync } from "node:fs";
10      import { join } from "node:path";
11
12      const LOCALES_DIR = join(import.meta.dirname, "locales");
13      const SRC_DIR = join(import.meta.dirname, "..");
14
15      function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
16        const keys: string[] = [];
17        for (const [k, v] of Object.entries(obj)) {
18          const key = prefix ? `${prefix}.${k}` : k;
19          if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
20          else keys.push(key);
21        }
22        return keys.sort();
23      }
24
25      const loadKeys = (locale: string): string[] =>
26        flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
27
28      describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
29        const enKeys = loadKeys("en");
```

```
30        const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
31          .filter((d) => d.isDirectory() && d.name !== "en")
32          .map((d) => d.name);
33
34        it("en is the non-empty source catalog", () => {
35          expect(enKeys.length).toBeGreaterThan(20);
36        });
37
38        for (const loc of others) {
39          it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
40            const keys = loadKeys(loc);
41            const missing = enKeys.filter((k) => !keys.includes(k));
42            const extra = keys.filter((k) => !enKeys.includes(k));
43            expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });
44          });
45        }
46      });
47
48      describe("no-bare-strings ? user-facing component attributes go through t()", () => {
49        const FILES = [
50          "App.tsx",
51          "components/QueryBar.tsx",
52          "components/RallyGrid.tsx",
53          "components/SelectionFooter.tsx",
54          "components/CompileResult.tsx",
55          "components/LanguageSwitcher.tsx",
56          "components/IngestPanel.tsx",
57        ];
58        // A hardcoded placeholder / aria-label / title with letters is a bare-string
regression ? it
59        // must be an expression ({t(...)}), not a quoted literal.
60        const BARE_ATTR = /\b(?:placeholder|aria-label|title)\s*=\s*"[^"]*[A-Za-z]{2}[^"]*"/g;
61
62        for (const f of FILES) {
63          it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
64            const src = readFileSync(join(SRC_DIR, f), "utf8");
65            expect(src.match(BARE_ATTR) ?? []).toEqual([]);
66          });
67        }
68      });
69
70      describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
71        it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google
Fonts)", () => {
72          const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
73          expect(main).toMatch(/@fontsource-variable\/noto-sans-kannada/);
74          expect(main).toMatch(/@fontsource-variable\/noto-sans-devanagari/);
75        });
76        it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
77          const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
78          expect(css).toMatch(/Noto Sans Kannada Variable/);
79          expect(css).toMatch(/Noto Sans Devanagari Variable/);
80        });
81      });
82
83      // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
internet) ?
84      // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
CDN/Google Fonts.
85      describe("offline / self-contained ? no external runtime assets", () => {
86        const CDN = /googleapis|gstatic|fonts\.google|cdnjs|unpkg|jsdelivr|\bcdn\.[a-z]/i;
87        const FILES = [
88          "../index.html", // web/index.html
89          "main.tsx",
```

```
90          "index.css",
91        ];
92        for (const rel of FILES) {
93          it(`${rel}: no external font/CDN reference`, () => {
94            expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
95          });
96        }
97        it("web/index.html loads no absolute-URL <link>/<script src> (everything is
bundled/local)", () => {
98          const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
99          expect(html).not.toMatch(/(?:href|src)\s*=\s*["']https?:\/\//i);
100       });
101     });
102
```

18. user (2026-06-21T15:29:43.324Z)

```
1       // @vitest-environment jsdom
2       //
3       // The SPA had NO render tests before this (verified) ? this file adds the jsdom safety
net for the
4       // i18n scaffold (tracker B1): real copy renders from the catalog, the switcher persists
the choice,
5       // missing-locale falls back to English, and ICU plurals format. It mounts the real <App
/>.
6       import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
7       import { render, screen, cleanup, fireEvent } from "@testing-library/react";
8       import i18n, { LANG_STORAGE_KEY } from "./index";
9       import App from "../App";
10
11      beforeEach(async () => {
12        // No engine in the test env: stub fetch so App's getConfig()/listRallies effects
reject quietly
13        // (App handles this gracefully ? cfg stays null) instead of hitting the network.
14        vi.stubGlobal("fetch", vi.fn(() => Promise.reject(new Error("no engine in test"))));
15        window.localStorage.clear();
16        await i18n.changeLanguage("en"); // also guarantees init completed before the first
render
17      });
18
19      afterEach(() => {
20        cleanup();
21        vi.unstubAllGlobals();
22      });
23
24      describe("i18n scaffold (B1)", () => {
25        it("renders user-facing copy from the en catalog (not raw keys)", () => {
26          render(<App />);
27          expect(screen.getByRole("heading", { level: 1 }).textContent).toBe("Pick the rallies
for your reel");
28          expect(screen.getByRole("button", { name: "Select all" })).toBeTruthy();
29        });
30
31        it("B2: the extracted component strings render (no leftover raw keys across the
page)", () => {
32          const { container } = render(<App />);
33          expect(screen.getByRole("button", { name: "Apply" })).toBeTruthy(); // QueryBar
34          expect(screen.getByRole("button", { name: /Build reel/ })).toBeTruthy(); //
SelectionFooter
35          expect(container.textContent).toContain("rally length"); // <Trans> hero intro +
query support
36          expect(container.textContent).toMatch(/\d+ rallies selected · \d+:\d+ total/); //
ICU footer summary
37          expect(container.textContent).toMatch(/Rally \d+/); // RallyGrid card
38          // No untranslated catalog keys leaked into the DOM.
```

```
39          expect(container.textContent).not.toMatch(/\b(app|grid|footer|query|mode|result|buil
d)\.[a-zA-Z]+/);
40        });
41
42        it("offers a language switcher and persists the choice (persisted pref wins on next
visit)", () => {
43          render(<App />);
44          const knBtn = screen.getByRole("button", { name: "?????" });
45          fireEvent.click(knBtn);
46          expect(i18n.language).toBe("kn");
47          expect(window.localStorage.getItem(LANG_STORAGE_KEY)).toBe("kn");
48          expect(knBtn.getAttribute("aria-pressed")).toBe("true"); // highlights the chosen
locale
49          // Switching actually re-renders the page into Kannada (the kn catalog, B5).
50          expect(screen.getByRole("heading", { level: 1 }).textContent).toBe("????? ?????????
??????????? ?????");
51        });
52
53        it("B5: switching to Kannada renders the kn catalog (Kannada copy, not English)",
async () => {
54          await i18n.changeLanguage("kn");
55          render(<App />);
56          expect(screen.getByRole("heading", { level: 1 }).textContent).toBe("????? ?????????
??????????? ?????");
57          expect(screen.getByRole("button", { name: "????????" })).toBeTruthy(); // Apply ?
Kannada
58          expect(screen.getByRole("button", { name: "????????? ?????????" })).toBeTruthy(); //
Select all ? Kannada
59          expect(document.documentElement.lang).toBe("kn"); // <html lang> tracks the active
locale (a11y)
60        });
61
62        it("B6: switching to Hindi renders the hi catalog (extensibility proof ? just a
catalog)", async () => {
63          await i18n.changeLanguage("hi");
64          render(<App />);
65          expect(screen.getByRole("heading", { level: 1 }).textContent).toBe("???? ??? ?? ???
??????? ?????");
66          expect(screen.getByRole("button", { name: "???? ????" })).toBeTruthy(); // Apply ?
Hindi
67          expect(document.documentElement.lang).toBe("hi");
68        });
69
70        it("a locale with no catalog falls back to English (never a raw key)", async () => {
71          await i18n.changeLanguage("ta"); // ta has no catalog
72          render(<App />);
73          expect(screen.getByRole("heading", { level: 1 }).textContent).toBe("Pick the rallies
for your reel");
74        });
75
76        it("formats ICU plurals from the catalog", () => {
77          expect(i18n.t("selection.count", { count: 1 })).toBe("1 rally selected");
78          expect(i18n.t("selection.count", { count: 3 })).toBe("3 rallies selected");
79        });
80      });
81
82      // The owner's bar (#41): a user whose BROWSER/OS language is Kannada/Hindi must auto-
get the
83      // respective version. These prove the negotiation NOW at the detection level (B1, en-
only); the
84      // kn/hi CONTENT assertions ride the catalogs in B5/B6. See FRIENDLY_AND_MULTILINGUAL.md
§6.5 (N1?N9).
85      describe("browser-language negotiation (B1: N1/N3?N7)", () => {
86        const detectBases = () => {
87          const d = i18n.services.languageDetector.detect();
```

```
88          return (Array.isArray(d) ? d : [d]).filter(Boolean).map((l) =>
String(l).split("-")[0]);
89        };
90
91        it("N1/N5: a Kannada browser (kn-IN) is auto-detected as kn for a first-time visitor",
() => {
92          window.localStorage.clear(); // no persisted preference yet
93          vi.stubGlobal("navigator", { language: "kn-IN", languages: ["kn-IN", "en-IN"] });
94          expect(detectBases()).toContain("kn"); // the kn-IN region subtag normalizes to base
kn
95        });
96
97        it("N2: a Hindi browser (hi-IN) is auto-detected as hi (now a supported locale)", ()
=> {
98          window.localStorage.clear();
99          vi.stubGlobal("navigator", { language: "hi-IN", languages: ["hi-IN", "en-IN"] });
100         expect(detectBases()).toContain("hi");
101       });
102
103       it("N4: a persisted preference beats the browser language (returning user keeps their
choice)", () => {
104         window.localStorage.setItem(LANG_STORAGE_KEY, "kn");
105         vi.stubGlobal("navigator", { language: "en-US", languages: ["en-US"] });
106         expect(detectBases()[0]).toBe("kn");
107       });
108
109       it("N3: persisted pref and ?lang are negotiated BEFORE the browser language", () => {
110         const order = i18n.options.detection?.order ?? [];
111         expect(order.indexOf("localStorage")).toBeGreaterThanOrEqual(0);
112         expect(order.indexOf("localStorage")).toBeLessThan(order.indexOf("navigator"));
113         expect(order.indexOf("querystring")).toBeLessThan(order.indexOf("navigator"));
114       });
115
116       it("N6: an unsupported browser language (Tamil) serves English content ? never a
locale we can't render", async () => {
117         window.localStorage.clear();
118         await i18n.changeLanguage("ta"); // ta is not in SUPPORTED_LOCALES
119         render(<App />);
120         expect(screen.getByRole("heading", { level: 1 }).textContent).toBe("Pick the rallies
for your reel");
121       });
122
123       it("N7: a key with no catalog entry falls back to English (no raw key)", async () => {
124         await i18n.changeLanguage("ta"); // ta has no catalog ? keys resolve via
fallbackLng=en
125         expect(i18n.t("app.title")).toBe("Pick the rallies for your reel");
126         expect(i18n.t("bulk.selectAll")).toBe("Select all");
127       });
128     });
129
```

19. assistant (2026-06-21T15:29:47.673Z)

Now let me check how CompileResult is consumed (its parent) to confirm the BuildState transitions and that the building branch isn't passed unexpectedly, and verify whether the i18n test's "no raw keys" regex covers `result.*`. Let me find the parent and check the `useIngestJob` hook for the `progress` shape.

20. user (2026-06-21T15:29:49.489Z)

```
web\src\App.tsx:8:import { CompileResult, type BuildState } from "./components/CompileResult";
web\src\App.tsx:61:  const [build, setBuild] = useState<BuildState>({ status: "idle" });
web\src\App.tsx:123:    if (build.status === "building") return;
web\src\App.tsx:129:      setBuild({
web\src\App.tsx:135:    setBuild({ status: "building" });
```

```
web\src\App.tsx:147:      setBuild({ status: "done", url, filename });
web\src\App.tsx:154:      setBuild({ status: "error", error: errorText(t, e) });
web\src\App.tsx:262:        <CompileResult build={build} onDismiss={() => setBuild({ status:
"idle" })} />
web\src\App.tsx:271:        building={build.status === "building"}
web\src\lib\errorText.ts:2:// the ingest flow (IngestPanel) and the build flow (App.onBuild ?
CompileResult), so a 5xx / engine-
web\src\i18n\hygiene.test.ts:54:    "components/CompileResult.tsx",
web\src\components\a8.status.test.tsx:8:import { CompileResult } from "./CompileResult";
web\src\components\a8.status.test.tsx:24:describe("A8: build state shows status + reassurance
(CompileResult)", () => {
web\src\components\a8.status.test.tsx:26:    render(<CompileResult build={{ status: "building"
}} onDismiss={() => {}} />);
web\src\components\a8.status.test.tsx:33:    render(<CompileResult build={{ status: "done", url:
"/x", filename: "r.mp4" }} onDismiss={() => {}} />);
web\src\components\a8.status.test.tsx:40:    render(<CompileResult build={{ status: "error",
error: "boom" }} onDismiss={() => {}} />);
web\src\components\build-error.test.tsx:3:// A5 ? the BUILD path (App.onBuild ? CompileResult)
now uses the SAME friendly mapping as ingest
web\src\components\CompileResult.tsx:9:export type BuildState =
web\src\components\CompileResult.tsx:16:  build: BuildState;
web\src\components\CompileResult.tsx:20:export function CompileResult({ build, onDismiss }:
Props) {
web\src\components\CompileResult.tsx:22:  if (build.status === "idle") return null;
web\src\components\CompileResult.tsx:26:  if (build.status === "building") {
web\src\components\CompileResult.tsx:35:  const isError = build.status === "error";
web\src\components\CompileResult.tsx:45:          {build.status === "done" ? t("result.ready") :
t("result.failed")}
web\src\components\CompileResult.tsx:57:      {build.status === "done" ? (
```

21. assistant (2026-06-21T15:29:53.643Z)

There's an A8 test file. Let me read it, plus check the i18n test's raw-key regex includes `result` (it does ? line 39 lists `result`). Let me read the A8 test and the App.tsx region around build, and check the previous version of IngestPanel for the removed local Spinner.

22. user (2026-06-21T15:29:54.593Z)

```
1       // @vitest-environment jsdom
2       //
3       // A8 ? honest expectations for non-tech users on slow uplinks: background-processing
reassurance and
4       // status shown as icon+TEXT chips (never icon-only), on BOTH the ingest progress and
the build state.
5       import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
6       import { render, screen, cleanup, fireEvent, act } from "@testing-library/react";
7       import i18n from "../i18n";
8       import { CompileResult } from "./CompileResult";
9       import { IngestPanel } from "./IngestPanel";
10      import type { JobStatus } from "../api/types";
11
12      beforeEach(async () => {
13        window.localStorage.clear();
14        await i18n.changeLanguage("en");
15      });
16
17      afterEach(() => {
18        cleanup();
19        vi.restoreAllMocks();
20        vi.unstubAllGlobals();
21        vi.useRealTimers();
22      });
23
24      describe("A8: build state shows status + reassurance (CompileResult)", () => {
25        it("building ? a 'stitching?' chip + a reassurance line (and nothing to dismiss yet)",
```

```
() => {
26          render(<CompileResult build={{ status: "building" }} onDismiss={() => {}} />);
27          expect(screen.getByText("Stitching your reel?")).toBeTruthy();
28          expect(screen.getByText(/usually takes under a minute/i)).toBeTruthy();
29          expect(screen.queryByRole("button", { name: "Dismiss" })).toBeNull();
30        });
31
32        it("done ? a ? icon paired with the 'ready' text (icon+text, not icon-only)", () => {
33          render(<CompileResult build={{ status: "done", url: "/x", filename: "r.mp4" }}
onDismiss={() => {}} />);
34          const h = screen.getByRole("heading", { level: 2 });
35          expect(h.textContent).toContain("?");
36          expect(h.textContent).toContain("Your reel is ready");
37        });
38
39        it("error ? a ? icon paired with 'failed' + the message", () => {
40          render(<CompileResult build={{ status: "error", error: "boom" }} onDismiss={() =>
{}} />);
41          const h = screen.getByRole("heading", { level: 2 });
42          expect(h.textContent).toContain("?");
43          expect(screen.getByText("boom")).toBeTruthy();
44        });
45      });
46
47      describe("A8: ingest progress reassurance + status chips (IngestPanel)", () => {
48        const httpOk = (s: number, j: unknown) => ({ ok: s >= 200 && s < 300, status: s, json:
async () => j, text: async () => JSON.stringify(j) });
49        function fetchSeq(process: unknown, jobs: JobStatus[]) {
50          let i = 0;
51          return vi.fn(async (url: string | URL, init?: RequestInit) => {
52            const u = String(url);
53            if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
54            if (u.startsWith("/api/jobs/")) {
55              const b = jobs[Math.min(i, jobs.length - 1)];
56              i += 1;
57              return httpOk(200, b);
58            }
59            throw new Error(`unexpected fetch ${u}`);
60          }) as unknown as typeof fetch;
61        }
62
63        it("while processing ? the working chip + 'safe to wait' reassurance", async () => {
64          vi.useFakeTimers();
65          vi.stubGlobal("fetch", fetchSeq(httpOk(202, { job_id: "j", status: "queued" }), [
66            { job_id: "j", status: "running", progress: "segmenting" },
67          ]));
68          render(<IngestPanel onLoaded={vi.fn()} />);
69          fireEvent.change(screen.getByLabelText(/video location/i), { target: { value:
"gs://b/x.mp4" } });
70          fireEvent.click(screen.getByRole("button", { name: /process video/i }));
71          await act(async () => {
72            await vi.advanceTimersByTimeAsync(0); // submit ? polling
73          });
74          await act(async () => {
75            await vi.advanceTimersByTimeAsync(1000); // running tick ? progress word
76          });
77          expect(screen.getByText(/processing your video/i)).toBeTruthy();
78          expect(screen.getByText(/safe to wait/i)).toBeTruthy();
79        });
80
81        it("on failure ? a ? 'Couldn't process' status chip alongside the friendly message",
async () => {
82          vi.stubGlobal("fetch", vi.fn(async () => {
83            throw new Error("offline");
84          }) as unknown as typeof fetch);
```

```
85          render(<IngestPanel onLoaded={vi.fn()} />);
86          fireEvent.change(screen.getByLabelText(/video location/i), { target: { value:
"gs://b/x.mp4" } });
87          fireEvent.click(screen.getByRole("button", { name: /process video/i }));
88          expect(await screen.findByText("Couldn't process")).toBeTruthy();
89          expect(await screen.findByText(/is it running/i)).toBeTruthy();
90        });
91      });
92
```

23. user (2026-06-21T15:29:55.408Z)

```
118         return (id: number) => m.get(id) ?? 0;
119       }, [rallies]);
120
121       // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
122       const onBuild = async () => {
123         if (build.status === "building") return;
124         const cid = newCorrelationId();
125         const filename = reelFilename(reelName);
126         const ids = [...sel.selected];
127         if (!videoId) {
128           log("info", "build.no_video", { cid, selected: ids.length });
129           setBuild({
130             status: "error",
131             error: t("build.noVideo"),
132           });
133           return;
134         }
135         setBuild({ status: "building" });
136         log("info", "build.start", {
137           cid,
138           video_id: videoId,
139           filename,
140           selected: ids.length,
141           total_sec: Math.round(sel.totalSec),
142           below_floor: warning?.atRisk.length ?? 0,
143         });
144         try {
145           const res = await compileReel(videoId, ids, filename, cid);
146           const url = highlightUrl(videoId, filename);
147           setBuild({ status: "done", url, filename });
148           log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
149         } catch (err) {
150           // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
151           // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
152           const status = err instanceof ApiError ? err.status : undefined;
153           const e = classifyError(err);
154           setBuild({ status: "error", error: errorText(t, e) });
155           log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
156         }
157       };
158
159       // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
160       // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
161       // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
162       const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
```

24. assistant (2026-06-21T15:30:00.001Z)

Now let me verify the actual test suite passes, run tsc/build, and check the i18n raw-key regex against `ingest.*` keys (the regex on line 39 does NOT include `ingest` or `selection` or `errors`/`lang`/`bulk` ? let me think about this carefully). Let me run the full web test suite and type-check.

25. user (2026-06-21T15:30:07.764Z)

{"ts":"2026-06-21T15:30:07.217Z","level":"error","event":"api.network_error","cid":"491befb2-324c-4ab2-863f-7af7b4e34dd3","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B5: switching to Kannada renders the kn catalog (Kannada copy, not English)
{"ts":"2026-06-21T15:30:07.217Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

stdout | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6: switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)
{"ts":"2026-06-21T15:30:07.242Z","level":"info","event":"api.request","cid":"5f3115b4-7a20-4923-9e3c-9228655e35a4","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6: switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)
{"ts":"2026-06-21T15:30:07.277Z","level":"error","event":"api.network_error","cid":"5f3115b4-7a20-4923-9e3c-9228655e35a4","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6: switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)
{"ts":"2026-06-21T15:30:07.277Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

stdout | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T15:30:07.298Z","level":"info","event":"api.request","cid":"475297e7-4356-4c4d-a962-a34fcbf9711a","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T15:30:07.305Z","level":"error","event":"api.network_error","cid":"475297e7-4356-4c4d-a962-a34fcbf9711a","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T15:30:07.305Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

stdout | src/i18n/i18n.test.tsx > browser-language negotiation (B1: N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ? never a locale we can't render
{"ts":"2026-06-21T15:30:07.326Z","level":"info","event":"api.request","cid":"90deab3c-3702-4117-bfcb-450e6cb03d33","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1: N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ? never a locale we can't render
{"ts":"2026-06-

```
21T15:30:07.330Z","level":"error","event":"api.network_error","cid":"90deab3c-3702-4117-bfcb-
450e6cb03d33","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-
21T15:30:07.331Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error
calling GET /api/config"}

 ? src/i18n/i18n.test.tsx (13 tests)
722ms
   ? i18n scaffold (B1) > renders user-facing copy from the en
catalog (not raw keys)  362ms

 Test Files  15 passed (15)
      Tests  141 passed (141)
   Start at  21:00:03
   Duration  4.19s (transform 851ms, setup 249ms, collect 4.49s, tests 2.83s,
environment 15.40s, prepare 4.03s)
```

26. assistant (2026-06-21T15:30:11.308Z)

```
141 tests pass. Now type-check and build.
```

27. user (2026-06-21T15:30:18.444Z)

```
---TSC EXIT: 0---
```

28. user (2026-06-21T15:30:27.117Z)

```
vite v6.4.3 building for production...
transforming...
? 90 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html
  0.40 kB ? gzip:   0.27 kB
dist/assets/noto-sans-devanagari-latin-ext-wght-normal-CLkupaxV.woff2
 13.85 kB
dist/assets/noto-sans-kannada-latin-ext-wght-normal-MdBBF41x.woff2
 13.99 kB
dist/assets/noto-sans-devanagari-latin-wght-normal-Bxfr4UJH.woff2
 25.38 kB
dist/assets/noto-sans-kannada-latin-wght-normal-DeLZCgFT.woff2
 27.58 kB
dist/assets/noto-sans-kannada-kannada-wght-normal-DuwB-P8d.woff2
 90.80 kB
dist/assets/noto-sans-devanagari-devanagari-wght-normal-DWI_L5Cu.woff2
121.19 kB
dist/assets/index-XjvH8X09.css
 21.13 kB ? gzip:   4.88 kB
dist/assets/index-CKFRQ2vk.js
350.71 kB ? gzip: 110.34 kB
? built in 2.29s
---BUILD EXIT: 0---
```

29. assistant (2026-06-21T15:30:38.448Z)

```
All green: 141 tests, tsc clean, build clean. Now let me do the detailed correctness analysis on
each review axis.

For axis 3 (no raw keys), the i18n test's raw-key regex on line 39 is:
`/\b(app|grid|footer|query|mode|result|build)\.[a-zA-Z]+/`
```

This includes `result` but NOT `ingest`. Important: this regex runs against the rendered `<App />` which does NOT include the building/done/error states by default (build starts idle ? returns null). So the new `result.building`/`result.buildingReassure` keys never render in that test anyway. The A8 test (`a8.status.test.tsx`) is the one that actually renders these states and asserts the values. Let me verify the `ingest.statusFailed` and `ingest.reassure` ? these are under `ingest.*` which the i18n regex does NOT catch. But the A8 test asserts the rendered VALUES ("Couldn't process", "safe to wait"), so a key leak there would fail. Good coverage.

Let me double check one thing: the aria-live on the building card. In `CompileResult`, the building branch is its own `<section aria-live="polite">`, but the done/error branch is a separate `<section>` WITHOUT aria-live. Let me think about whether the announcement works when transitioning building ? done (the live region unmounts and a new non-live section mounts). Let me also confirm the `progress` field exists on the polling phase type.

30. user (2026-06-21T15:30:38.780Z)

```
1       // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2       // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3       //
4       // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5       // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6       // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7       // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8       // cancellable on unmount / user-cancel / a superseding submit.
9       import { useCallback, useEffect, useRef, useState } from "react";
10      import { getJob, processVideo } from "../api/client";
11      import { classifyError, type IngestError } from "../lib/errors";
12      import { log, newCorrelationId } from "../lib/log";
13      import type { JobStatus } from "../api/types";
14
15      const POLL_MIN_MS = 1000;
16      const POLL_MAX_MS = 5000;
17      const POLL_BACKOFF = 1.5;
18      const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20      export type IngestPhase =
21        | { status: "idle" }
22        | { status: "submitting" }
23        | { status: "polling"; progress: string | null }
24        | { status: "done"; videoId: string }
25        | { status: "error"; error: IngestError };
26
27      interface Run {
28        cancelled: boolean;
29        timer?: ReturnType<typeof setTimeout>;
30      }
31
32      export function useIngestJob(onLoaded?: (videoId: string) => void) {
33        const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34        const runRef = useRef<Run | null>(null);
35        // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36        const onLoadedRef = useRef(onLoaded);
37        onLoadedRef.current = onLoaded;
38
39        const stop = useCallback(() => {
40          const run = runRef.current;
41          if (run) {
42            run.cancelled = true;
```

```
43            if (run.timer) clearTimeout(run.timer);
44          }
45          runRef.current = null;
46        }, []);
47
48        // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
flag idiom).
49        useEffect(() => stop, [stop]);
50
51        const submit = useCallback(
52          (rawUri: string) => {
53            const uri = rawUri.trim();
54            if (!uri) {
55              setPhase({ status: "error", error: { code: "invalidUri" } });
56              return;
57            }
58
59            stop(); // supersede any prior run
60            const run: Run = { cancelled: false };
61            runRef.current = run;
62
63            const cid = newCorrelationId();
64            const startedAt = Date.now();
65            setPhase({ status: "submitting" });
66            log("info", "ingest.submit_start", { cid, uri });
67
68            const fail = (error: IngestError, event: string) => {
69              if (run.cancelled) return;
70              setPhase({ status: "error", error });
71              const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
72              log(level, event, { cid, uri, code: error.code, detail: error.detail });
73            };
74
75            const poll = (jobId: string, delay: number) => {
76              run.timer = setTimeout(() => {
77                void (async () => {
78                  if (run.cancelled) return;
79                  let job: JobStatus;
80                  try {
81                    job = await getJob(jobId, cid);
82                  } catch (err) {
83                    fail(classifyError(err), "ingest.poll_error");
84                    return;
85                  }
86                  if (run.cancelled) return;
87
88                  if (job.status === "done") {
89                    const videoId = job.video_id ?? job.result?.video_id ?? null;
90                    // Two status layers: a `done` job with a `failed` pipeline verdict (or no
video_id) is
91                    // a success-shaped FAILURE ? never load it, never hang.
92                    if (job.result?.status === "failed" || !videoId) {
93                      fail({ code: "emptyResult" }, "ingest.empty_result");
94                      return;
95                    }
96                    setPhase({ status: "done", videoId });
97                    log("info", "ingest.done", { cid, uri, video_id: videoId });
98                    onLoadedRef.current?.(videoId);
99                    return;
100                 }
101                 if (job.status === "failed") {
102                   fail({ code: "engine", detail: job.error ?? "job failed" },
"ingest.job_failed");
103                   return;
```

```
104                 }
105                 // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106                 setPhase({ status: "polling", progress: job.progress ?? null });
107                 if (Date.now() - startedAt > POLL_CEILING_MS) {
108                   fail({ code: "timeout" }, "ingest.poll_timeout");
109                   return;
110                 }
111                 poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112               })();
113             }, delay);
114           };
115
116           void (async () => {
117             let jobId: string;
118             try {
119               const resp = await processVideo(uri, cid);
120               jobId = resp.job_id;
121             } catch (err) {
122               fail(classifyError(err), "ingest.submit_error");
123               return;
124             }
125             if (run.cancelled) return;
126             log("info", "ingest.queued", { cid, uri, job_id: jobId });
127             setPhase({ status: "polling", progress: null });
128             poll(jobId, POLL_MIN_MS);
129           })();
130         },
131         [stop],
132       );
133
134       const cancel = useCallback(() => {
135         stop();
136         setPhase({ status: "idle" });
137         log("info", "ingest.cancel", {});
138       }, [stop]);
139
140       const reset = useCallback(() => {
141         stop();
142         setPhase({ status: "idle" });
143       }, [stop]);
144
145       const busy = phase.status === "submitting" || phase.status === "polling";
146       return { phase, submit, cancel, reset, busy };
147     }
148
```

31. assistant (2026-06-21T15:30:46.354Z)

I now have everything to confirm correctness. Let me verify the `IngestPanel` progress narrowing: line 80 uses `phase.status === "polling" && phase.progress`. `busy` is true for `submitting` OR `polling`. During `submitting`, `phase.progress` doesn't exist on the type ? but the `phase.status === "polling"` guard narrows correctly, so TS is happy (and tsc passed). Good.

Let me check the `ingest.progress` key. The working chip label appends `· ${t("ingest.progress", { progress: phase.progress })}`. The `ingest.progress` value is just `"{progress}"` ? so it interpolates the raw engine phase word (e.g. "segmenting"). This is the engine's free-text phase, not localized, which matches the design comment. Not a raw-i18n-key issue.

Now let me confirm there's no dangling local Spinner in IngestPanel (it imports from StatusChip now, line 12) and grep for any other local Spinner definitions.

32. user (2026-06-21T15:30:48.135Z)

```
web\src\components\StatusChip.tsx:33:export function Spinner() {
```

```
web\src\components\StatusChip.tsx:36:      className="inline-block h-3.5 w-3.5 animate-spin
rounded-full border-2 border-ink/20 border-t-ink/70"
```

33. assistant (2026-06-21T15:30:51.838Z)

Only one Spinner definition (in StatusChip), no dangling local copy. Let me confirm the git diff to see what changed (verify the local Spinner was actually removed from IngestPanel and confirm exactly which files this PR touches).

34. user (2026-06-21T15:30:52.901Z)

(Bash completed with no output)

35. user (2026-06-21T15:30:58.928Z)

```
---DIFF STAT---
```

36. assistant (2026-06-21T15:31:02.737Z)

The branch has no commits yet beyond origin/master ? the changes are in the working tree (uncommitted). Let me confirm the working-tree state.

37. user (2026-06-21T15:31:03.891Z)

```
 M web/src/components/CompileResult.tsx
 M web/src/components/IngestPanel.tsx
 A web/src/components/StatusChip.tsx
 A web/src/components/a8.status.test.tsx
 M web/src/i18n/locales/en/common.json
 M web/src/i18n/locales/hi/common.json
 M web/src/i18n/locales/kn/common.json
?? .claude/
?? site/access.db
---HEAD---
501c014 feat(web): A5 ? friendly engine-error mapping across build + load (shared errors.*)
(#62)
```

38. assistant (2026-06-21T15:31:07.656Z)

The change set matches the described scope exactly. Let me verify the removed local Spinner by diffing IngestPanel against HEAD, and confirm what the old version looked like.

39. user (2026-06-21T15:31:08.481Z)

```
diff --git a/web/src/components/IngestPanel.tsx b/web/src/components/IngestPanel.tsx
index 87607f8..6f0c37f 100644
--- a/web/src/components/IngestPanel.tsx
+++ b/web/src/components/IngestPanel.tsx
@@ -9,6 +9,7 @@ import type { FormEvent } from "react";
 import { useTranslation } from "react-i18next";
 import { useIngestJob } from "../hooks/useIngestJob";
 import { errorText } from "../lib/errorText";
+import { StatusChip, Spinner } from "./StatusChip";

 interface Props {
   /** Called with the engine's video_id once a submitted job finishes successfully. */
@@ -66,23 +67,29 @@ export function IngestPanel({ onLoaded }: Props) {

       <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>

-      {/* Live status (aria-live so screen readers hear progress). Indeterminate: the engine
reports
-          phase words ("hashing", "segmenting"), not a percentage ? so a spinner + the phase
word. */}
```

```
+      {/* Live status (aria-live so screen readers hear progress) as icon+text chips ? never
icon-only
+          (A8). Indeterminate: the engine reports phase words ("hashing", "segmenting"), not a
%. */}
       <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
         {busy && (
-          <p className="flex items-center gap-2 text-ink/70">
-            <Spinner />
-            <span>
-              {t("ingest.working")}
-              {phase.status === "polling" && phase.progress
-                ? ` · ${t("ingest.progress", { progress: phase.progress })}`
-                : ""}
-            </span>
-          </p>
+          <div>
+            <StatusChip
+              tone="info"
+              icon={<Spinner />}
+              label={
+                t("ingest.working") +
+                (phase.status === "polling" && phase.progress
+                  ? ` · ${t("ingest.progress", { progress: phase.progress })}`
+                  : "")
+              }
+            />
+            {/* Reassurance for slow uplinks: the job runs on the server, so it's safe to wait.
*/}
+            <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
+          </div>
         )}
         {phase.status === "error" && (
           <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-red-800"
data-testid="ingest-error">
-            <p>{errorMsg}</p>
+            <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
+            <p className="mt-2">{errorMsg}</p>
             <button
               type="button"
               onClick={reset}
@@ -96,12 +103,3 @@ export function IngestPanel({ onLoaded }: Props) {
     </section>
   );
 }
-
-function Spinner() {
-  return (
-    <span
-      className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-ink/20
border-t-ink/70"
-      aria-hidden="true"
-    />
-  );
-}
```

40. assistant (2026-06-21T15:31:19.843Z)

The local Spinner was cleanly removed and replaced by the import. Now I have a complete picture. Let me do a final careful analysis on the aria-live correctness for both regions, which is the subtle a11y point.

IngestPanel aria-live region (line 72): The `<div aria-live="polite" data-testid="ingest-status">` is a STABLE container that is always mounted (it's not conditionally rendered). The chips appear/change inside it. This is the correct pattern ? a live region must exist in the DOM before its content changes for SR to announce mutations. Both the working chip and error chip

swap inside this stable region. Correct. The StatusChip's text label is a real `<span>{label}</span>` so the announced text is meaningful ("Processing your video?", "Couldn't process"). The icon spans are `aria-hidden`. Good.

CompileResult building card (line 28): Here `aria-live="polite"` is on the building `<section>` itself, which is conditionally mounted (returns null on idle, separate section on building, another separate section on done/error). This is a subtle a11y weakness: when the region is mounted *together with* its content (rather than the content changing inside an already-present live region), some screen readers do NOT announce it because the live region wasn't present at the prior render to observe a mutation. Also, the done/error section has no aria-live, so the building?done transition (live section unmounts, non-live section mounts) is the least reliable announcement. This is a real but minor a11y nuance worth flagging.

Let me verify the done/error section indeed lacks aria-live (confirmed from the read: line 37-40 section has no aria-live). And confirm CompileResult is rendered unconditionally by the parent (so it's the component returning null, not the parent gating mount).

41. user (2026-06-21T15:31:21.227Z)

```
255               <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
256             </div>
257
258             <div className="mt-5">
259               <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
260             </div>
261
262             <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
263           </section>
264
265           <SelectionFooter
266             count={sel.count}
267             totalSec={sel.totalSec}
268             reelName={reelName}
269             onReelNameChange={setReelName}
270             onBuild={onBuild}
271             building={build.status === "building"}
272             warning={warning}
273           />
274         </main>
```

42. assistant (2026-06-21T15:31:29.679Z)

`<CompileResult>` is rendered unconditionally; the component itself returns null on idle. The `data-testid="compile-result"` is on the building section AND on the done/error section (both have it) ? so a test can find the card in both states.

I have everything I need for a complete, precise review. Let me compile findings.

Summary of verification:
- 141 web tests green (reproduced locally), tsc clean (exit 0), build clean (exit 0).
- Key parity: all 4 keys (`ingest.reassure`, `ingest.statusFailed`, `result.building`, `result.buildingReassure`) present in en/kn/hi ? no asymmetry.
- Icon+text: every status pairs an aria-hidden icon with a meaningful text label; no icon-only indicators; color is never the sole signal.
- No raw keys: A8 test asserts rendered VALUES; i18n "no raw keys" regex doesn't render these states but A8 covers them.
- Regression: idle ? null preserved; building branch has no dismiss button; done renders video+download; error renders message; TS narrowing clean; local Spinner fully removed.

The only substantive finding is the minor aria-live nuance in CompileResult.

43. user (2026-06-21T15:31:44.550Z)

```
Output does not match required schema: root: must have required property 'findings'
```

44. assistant (2026-06-21T15:31:49.643Z)

```
The tool call seems to have been malformed. Let me retry with the correct shape.
```

45. user (2026-06-21T15:31:59.649Z)

```
Output does not match required schema: root: must have required property 'findings'
```

46. assistant (2026-06-21T15:32:06.025Z)

```
The schema error persists despite providing both properties. The validator may be rejecting one
of the finding objects. Let me retry with a minimal, strictly-conformant payload to isolate the
issue.
```

47. user (2026-06-21T15:32:14.002Z)

```
Output does not match required schema: root: must have required property 'findings'
```

48. assistant (2026-06-21T15:32:19.156Z)

```
The error references `root` missing `findings` even though I'm passing it. This suggests the
harness may be receiving an empty/malformed args object ? possibly an encoding issue with a long
single-line `findings` value. Let me try with a much shorter detail string to rule out
length/escaping.
```

49. user (2026-06-21T15:32:24.015Z)

```
Output does not match required schema: root: must have required property 'findings'
```

50. assistant (2026-06-21T15:32:30.422Z)

```
Still failing. The validator complaint that `root` lacks `findings` while I clearly pass it
strongly suggests the JSON parse of my arguments is failing (likely a character in the string is
breaking the args envelope), causing the harness to see an empty object. Let me try the absolute
minimum: a summary plus an empty findings array, to confirm the call mechanism works at all.
```

51. user (2026-06-21T15:32:32.743Z)

```
Output does not match required schema: root: must have required property 'findings'
```

52. assistant (2026-06-21T15:32:38.674Z)

```
Even an empty array fails with the same `root must have required property findings` error. This
is not about my content ? the harness is reporting that `root` (the top-level object) lacks
`findings`, which means my arguments object is not reaching the validator intact. This points to
a harness-side parsing problem with this particular tool invocation in this turn.

Since the StructuredOutput tool is not accepting my payload despite a schema-conformant shape,
I'll record my complete findings here as text so they are not lost, and continue attempting the
tool. Let me try once more with both required keys ordered explicitly and no array nesting
issues.
```

53. user (2026-06-21T15:32:39.196Z)

```
Structured output provided successfully
```

54. assistant (2026-06-21T15:32:49.771Z)

```
The minimal call succeeded ? the issue was payload size/content in the longer attempts, not the
```

schema. Let me now submit the real findings with the strings trimmed to safe lengths.

55. user (2026-06-21T15:32:59.642Z)

Output does not match required schema: root: must have required property 'findings'

56. assistant (2026-06-21T15:33:03.467Z)

The long summary fails but the minimal one succeeded. There's a size threshold on the combined payload. Let me drastically shorten both the summary and the finding.

57. user (2026-06-21T15:33:08.166Z)

Output does not match required schema: root: must have required property 'findings'

58. assistant (2026-06-21T15:33:12.244Z)

Still failing at this size, yet the two-field minimal call worked. Let me test which field is the culprit by sending a short summary with the full finding, then vice versa.

59. user (2026-06-21T15:33:13.971Z)

Structured output provided successfully

60. assistant (2026-06-21T15:33:30.074Z)

That succeeded. The earlier failures were caused by the long `summary` field. The structured output is now recorded with my full finding. The short summary plus the complete finding went through.

To make sure the key verification points are not lost, here is my complete review (already captured in the StructuredOutput finding, expanded here for clarity):

Verification (all reproduced locally): 141 web tests pass, `tsc --noEmit` exits 0, `npm run build` is clean.

1. Icon+text / a11y ? PASS. Every status pairs an `aria-hidden` icon with a real text label: `StatusChip` renders `<span aria-hidden="true">{icon}</span>` + `<span>{label}</span>` (StatusChip.tsx:23-26); `Spinner` is `aria-hidden` (line 38). IngestPanel uses StatusChip for both the working chip (Spinner + "Processing your video?") and the error chip ("?" + `ingest.statusFailed`). CompileResult's done/error header pairs an `aria-hidden` ?/? with `result.ready`/`result.failed` (CompileResult.tsx:44-45). No icon-only indicators; color is never the sole signal. The IngestPanel `aria-live` region (line 72) is a stable always-mounted container ? the correct pattern.

2. Key-parity ? PASS. All four keys (`ingest.reassure`, `ingest.statusFailed`, `result.building`, `result.buildingReassure`) exist in en, kn, and hi with no asymmetry. The hygiene test stays green.

3. No raw keys ? PASS. `a8.status.test.tsx` asserts rendered VALUES ("Stitching your reel?", "usually takes under a minute", "Couldn't process", "safe to wait"), so a key leak fails there. The i18n.test.tsx "no raw keys" regex includes `result` but never exercises the building/done/error states (build starts idle ? CompileResult returns null), so it neither catches nor false-positives on these.

4. Regression ? PASS. Idle returns null (line 22); building branch has no dismiss button (verified by the A8 test `queryByRole("button", {name: "Dismiss"})` being null); done renders video+download; error renders the message; TS narrowing is valid (tsc clean); the local Spinner was fully removed from IngestPanel and replaced by the import ? only one Spinner definition remains (StatusChip.tsx:33).

One minor finding: aria-live placement in CompileResult is less robust than IngestPanel's (it's on the conditionally-mounted building section, and the done/error section has none), so the terminal "ready/failed" status may not be announced by screen readers. Not a blocker ? A8

tests assert text presence, not announcement.

Relevant files (absolute paths):
- `C:\Users\avidu\Projects\khelsutra\web\src\components\StatusChip.tsx`
- `C:\Users\avidu\Projects\khelsutra\web\src\components\IngestPanel.tsx`
- `C:\Users\avidu\Projects\khelsutra\web\src\components\CompileResult.tsx`
- `C:\Users\avidu\Projects\khelsutra\web\src\components\a8.status.test.tsx`
- `C:\Users\avidu\Projects\khelsutra\web\src\i18n\locales\{en,kn,hi}\common.json`